Source: https://codetrappers.com/wordpress-mcp-ai-agents-manage-content-woocommerce-workflows/

WordPress + MCP: How AI Agents Can Manage Content, WooCommerce and Business Workflows

The Abilities API is in WordPress core and the MCP Adapter connects Claude and ChatGPT to it. What ships today, what lands in 7.2, and the guardrails production needs.

WordPress has grown a standard way to describe what it can do and a standard way to let an agent do it. Here is what has actually shipped, what has not, and what we learned running an MCP bridge against our own production site.

Yusuf Mudagal 9 Sep 2026 13 min read

For three years the answer to “can we add AI to our WordPress site” has been a chat bubble in the corner. It answers questions about your content. It cannot do anything.

That is changing, and not because the models got better. It is changing because WordPress has grown a standard way to describe what it can do, and a standard way to let an outside agent do it. The first is the Abilities API, which has been in core since WordPress 6.9. The second is the Model Context Protocol (https://codetrappers.com/custom-mcp-server-development/), and there is now an official WordPress MCP Adapter that exposes abilities to agents such as Claude and ChatGPT.

Put together, they change what a WordPress site is. It stops being only a place where people log in and click, and becomes something an agent can be pointed at and told what you want.

We have been running this against our own production site rather than a sandbox. Most of this article is what that actually taught us, including the parts that are not ready.

What has actually shipped, and what has not

There is a lot of enthusiastic writing about this that describes 2027 as though it were today. Here is the honest state of it as of September 2026.

Piece

Status

Abilities API

In core. Landed in WordPress 6.9, gained a client-side JavaScript counterpart in 7.0, and in 7.1 wp_get_abilities() gained filtering by category, namespace and metadata. The canonical feature plugin was archived in February 2026 because it had been merged.

MCP Adapter

A separate plugin, still pre-1.0. It is on GitHub under the WordPress organisation

and was being reviewed for submission to the plugin directory in early September 2026. It has already had breaking changes at 0.3.0 and 0.5.0.

AI plugin

A separate canonical plugin, version 1.3.0. It ships abilities such as core/read-content and core/read-users, and since 1.3.0 those are behind a single opt-in setting rather than registered automatically.

Create, update and delete abilities

Not yet. Standardised management abilities are planned for the WordPress 7.2 cycle, which is in development now and runs to around December 2026.

Embeddings and semantic search (https://codetrappers.com/ai-search-knowledge-assistants/)

Not in core. It missed the 7.1 beta cutoff, and the current direction is to build storage and similarity inside the AI plugin rather than add a core database table.

The short version: the plumbing that lets an agent discover and read is real and in core. The standardised plumbing that lets it write is a cycle away. Anyone selling you a finished, officially supported “AI manages your WordPress” product today is selling you a custom build with a coat of paint on it.

That is not a reason to wait. It is a reason to know which parts you are building yourself, and to build those parts so they survive the standard arriving.

How the pieces fit together

Three layers, and it is worth being precise about which does what, because the names get used interchangeably.

1. An ability is a described capability

An ability is a named thing your site can do, registered with a schema for its input and output and a callback that decides who is allowed to run it. Registration hooks into wp_abilities_api_init:

```
add_action( 'wp_abilities_api_init', function () {
    wp_register_ability( 'codetrappers/find-orders-on-hold', array(
        'label'               => 'Find orders on hold',
        'description'         => 'Returns WooCommerce orders stuck on-hold for longer
than N days.',
        'input_schema'        => array(
            'type'       => 'object',
            'properties' => array(
                'days' => array( 'type' => 'integer', 'minimum' => 1, 'maximum' =>
90 ),
            ),
            'required'   => array( 'days' ),
        ),
        'execute_callback'    => 'codetrappers_find_orders_on_hold',
        'permission_callback' => function () {
            return current_user_can( 'edit_shop_orders' );
        },
```

```
    ) );
} );
```

Two things in there matter more than they look. The input schema is not documentation, it is a contract the agent is held to, so a model cannot invent a parameter or pass ninety thousand days. And the permission callback runs on every single call, which means an ability is bounded by the same capability system that has protected WordPress for twenty years, not by a prompt asking the model to behave.

2. MCP is how an outside agent reaches them

The MCP Adapter exposes registered abilities over the Model Context Protocol, at a REST endpoint shaped like /wp-json/mcp/your-server-name. It also supports STDIO through WP-CLI, and a transport interface if you need something else.

Abilities are private by default. One has to be explicitly marked public, and authentication happens twice over: once at the transport, and again per ability through that permission callback. Exposure is opt-in at both levels, which is the correct default and the opposite of how most integrations start life.

3. The agent plans, the site executes

The model never touches your database. It sees a list of tool descriptions, decides which to call and with what arguments, and your PHP does the work under your user’s permissions. That separation is the entire security story, and it is why “can the AI break my site” has a better answer here than in a plugin that pipes model output into eval.

What this looks like in practice

Some concrete things we have done through this bridge on our own site, to show the shape of it rather than to impress anyone.

Publishing a set of related pages. We had six migration (https://codetrappers.com/migration-for-wordpress/) landing pages driven by one template and an array keyed by slug. Adding four more platforms meant four array entries, four pages created with the right template and meta, and the homepage links repointed. As an instruction that is one sentence. As clicking it is about forty minutes of repetitive work with three places to make a typo.

Fixing a bug found by measurement rather than by reading. Our site search was returning nothing for any question containing a three digit number. The cause was array_keys() handing back integers, because PHP casts numeric strings when they are used as array keys, and a downstream str_replace() then threw. Finding that meant querying live data, not reading code. An agent with read access to the site found it in one pass.

Reordering a menu without breaking the rest of it. A mega menu renders in menu_order, so reassigning positions can silently shove another column’s items around. The fix was to reuse only positions the affected items already owned, and to refuse to write any other. That constraint was expressed once, in code, and then enforced on every run.

Answering “what did people actually ask?” Reading a live table, grouping it and summarising it is a five second job for an agent and a ticket for a developer.

The pattern in all four is the same. These are not creative tasks. They are tasks where the knowledge required is spread across a database, a theme, a plugin and a live page, and where a human is slow mostly because they have to go and look in four places.

WooCommerce is where this earns its keep

Content editing is the obvious demo. Commerce is the one with money attached.

Useful abilities on a store, roughly in order of how quickly they pay for themselves:

- **Find the stuck things.** Orders on hold past a threshold, failed payments, subscriptions about to lapse, products that have been out of stock for a month. These are queries someone should run every morning and nobody does.
- **Bulk catalogue work with judgement in it.** “Every product in this category is missing a meta description, write one from the product data and flag anything where the data is too thin to describe honestly.” The flagging is the valuable half.
- **Pricing and stock reconciliation.** Compare what the site says against what the supplier feed says and report the differences, without changing anything.
- **Customer enquiry triage.** Match an inbound question to an order, a shipment and a product, so a human opens one screen instead of four.

Notice how many of those are read-only. That is deliberate, and it is the recommendation: start with abilities that only look. A read-only agent that surfaces the right five things every morning is worth more than a write-enabled one you are frightened to leave alone, and it is how you build the confidence to grant more later.

The part the demos skip: blast radius

Every impressive MCP demo shows an agent doing something. Almost none show what happens when it does the wrong thing, and that is the part that decides whether this belongs anywhere near a production site.

These are the guardrails we actually use, all of which came from something going wrong first.

- **Verify before overwriting, always.** Every file write is checksummed before it lands, and the write is refused if the content does not match what was intended. Content that travels through several layers can arrive subtly altered, and a silent partial write to a stylesheet is a blank site.
- **Lint before deploying, never after.** Every PHP file is passed through php -l before it replaces anything. A syntax error deployed to a live theme is a fatal error on every page, and the agent that caused it has just removed its own means of fixing it.
- **Back up in the same operation as the write.** Not as a separate step that can be skipped when things feel routine.
- **Validate the result, not the intention.** We minify CSS on this site. The first version silently swallowed 98KB because an apostrophe inside an English comment

opened a string that never closed. The fix was not better regex, it was counting braces before and after and refusing to ship if the number changed. Check the output, because the code that produced it always believes it worked.

- **Make dangerous positions unreachable rather than trusting care.** The menu example above: rather than being careful about which positions to write, the code refuses to write a position that is not already owned by the item being moved.
- **Approval gates on anything outward-facing.** Publishing, emailing, charging, deleting. An agent should draft those and stop.

The honest summary is that an agent operating a production site is a junior developer with unlimited energy, no fear, and no instinct for which mistakes are recoverable. You do not solve that with a better prompt. You solve it with the same things that make human deployment safe: verification, linting, backups (https://codetrappers.com/maintenance-for-wordpress/), staged permissions, and a review step before anything reaches the public.

Semantic search, and what to do while embeddings land

Embeddings are the missing piece for “answer questions about my own content”, and they are not in core. The current plan is to build vector storage and similarity inside the AI plugin rather than as a core database table, and it slipped out of 7.1.

You do not have to wait, and you should not build your own vector database either. In between there is a retrieval system that indexes your content into a table, scores passages against a question, and hands the top matches to a model as context. That is retrieval-augmented generation without the fashionable parts, and on a site of a few hundred pages it performs well.

Two lessons from running one on this site. First, retrieval quality is a content problem before it is a maths problem: our portfolio pages were invisible to search because the words visitors use, “portfolio” and “case study”, appeared nowhere in the indexed text. Adding that vocabulary to the index fixed what no amount of tuning would have. Second, the failure to design for is not a wrong answer, it is a confident answer drawn from an article that happens to sit on your site and quotes somebody else’s prices. The system prompt has to draw that line explicitly.

Build the retrieval layer so the indexing is separable from the scoring. When core embeddings arrive, you replace one half and keep the rest.

Connecting the rest of the business

MCP is a protocol, not a WordPress feature, so the same agent that reads your site can hold connections to other systems at the same time. That is where the workflow value sits: not “AI writes a blog post”, but “when a WooCommerce (https://codetrappers.com/wordpress-for-ecommerce/) order fails twice, look up the customer in the CRM, check whether a support ticket already exists, and draft the follow-up”.

Practical notes, since this is the area where enthusiasm outruns reality fastest:

- **Tools such as n8n are the right home for the deterministic part.** Triggers, retries, scheduling and error handling are solved problems. Put the “if this then that” in the workflow engine and reserve the agent for the step that genuinely needs judgement.
- **WhatsApp Business and most CRMs have their own approval and rate limits.** An agent that can message customers directly is an agent that can message a thousand customers at three in the morning. Drafts and a queue.
- **Log everything the agent did, in a form a person can read later.** When something odd happens in a month, “which tool ran with which arguments” is the only question that matters, and it cannot be answered retrospectively.

Where this is going

WordPress 7.2 is in development now, with a release squad announced in September 2026 and a cycle running to around December. Standardised management abilities, the create, update and delete side, are targeted at that cycle. The MCP Adapter is heading for the plugin directory. Embeddings are being built inside the AI plugin.

When those land, a large amount of what is currently custom becomes standard, and the sites that benefit immediately will be the ones whose functionality is already described as abilities with real permission callbacks. That work is not speculative. It is good practice regardless, because an ability is just a well-defined, permission-checked function with a schema, which is what that code should have been anyway.

The sites that will struggle are the ones where the business logic lives in a page builder, in a settings screen, or in a plugin nobody can read. You cannot expose to an agent what you cannot describe.

What we build

We are a WordPress engineering agency in Bangalore, and this is not a side interest for us. Our own production site is operated through an MCP bridge, which is where every example and every guardrail above comes from.

For clients that usually means one of four things:

- **Describing your site as abilities.** Auditing what your site actually does, and exposing the parts worth exposing with proper permission callbacks and schemas.
- **A private agent bridge.** MCP set up against your own site, with authentication, per-ability permissions, an audit log and approval gates, so your team can use Claude or ChatGPT against your real data without that data leaving your control.
- **Retrieval and semantic search over your content.** Built so the indexing layer can be swapped for core embeddings when they arrive.
- **Workflow integration.** WordPress and WooCommerce joined to the CRM, the messaging channel and the automation layer, with the deterministic parts in a workflow engine and the judgement parts, and only those, given to an agent.

If you have a WordPress or WooCommerce site and a process that eats hours every week, that is the conversation worth having. Not “can AI write our blog posts”, but

“which of these jobs is really a query nobody has time to run”.

Talk to an engineer (https://codetrappers.com/contact/) and tell us the process. We will tell you honestly whether it is ready to be automated today, or whether it is one of the things worth waiting a release for.

Yusuf Mudagal

Founder, CodeTrappers

- Yusuf Mudagal on LinkedIn (https://www.linkedin.com/in/yusuf-mudagal/)
- Yusuf Mudagal on X (https://x.com/YusufMudagal)
- Yusuf Mudagal on GitHub (https://github.com/yusufhay)
- Yusuf Mudagal on WordPress.org profile (https://profiles.wordpress.org/yusufmudagal/)
- Yusuf Mudagal on Email

Read 132 other articles by this author (https://codetrappers.com/author/yusuf-mudagal/)